

Last updated Nov 14, 2025

Worms

1ª fase do projeto de Laboratórios de Informática / Programação I
2025/26

Introdução

Neste enunciado apresentam-se as tarefas referentes à primeira fase do projeto das unidades curriculares de Laboratórios de Informática I e Laboratórios de Programação I. O projeto será desenvolvido por grupos de 2 elementos, e consiste em pequenas aplicações Haskell que deverão responder a diferentes tarefas (apresentadas adiante).

O objetivo do projeto deste ano é implementar uma variante da famosa saga de jogos de estratégia Worms. A ideia geral do jogo é, controlando uma ou mais minhocas com um arsenal de armas à sua disposição, eliminar as minhocas inimigas antes de se ser eliminado. O jogo original é baseado em equipas e turnos, embora isso não seja diretamente relevante para as tarefas propostas.



Tarefas

Importante: Cada tarefa deverá ser desenvolvida num módulo Haskell independente, nomeado `Tarefan.hs`, em que `n` é o número da tarefa. Cada grupo terá um repositório de código associado previamente inicializado com *stubs* dos ficheiros das tarefas. Os grupos **não devem alterar** os nomes, tipos e assinaturas das funções previamente definidas, sob pena de não serem corretamente avaliados.

Para aceder pela primeira vez ao repositório Git do seu grupo, pode executar o seguinte comando (substituindo `XXX` pelo número do seu grupo):

```
git clone git@gitlab.com:uminho-di/li1/2526/projetos/202511gXXX.git
```

O comando irá pedir para se autenticar. Para isso deve ter uma conta GitLab associada ao seu email institucional `aXXXXX@alunos.uminho.pt`.

Tipos de dados

Mapas

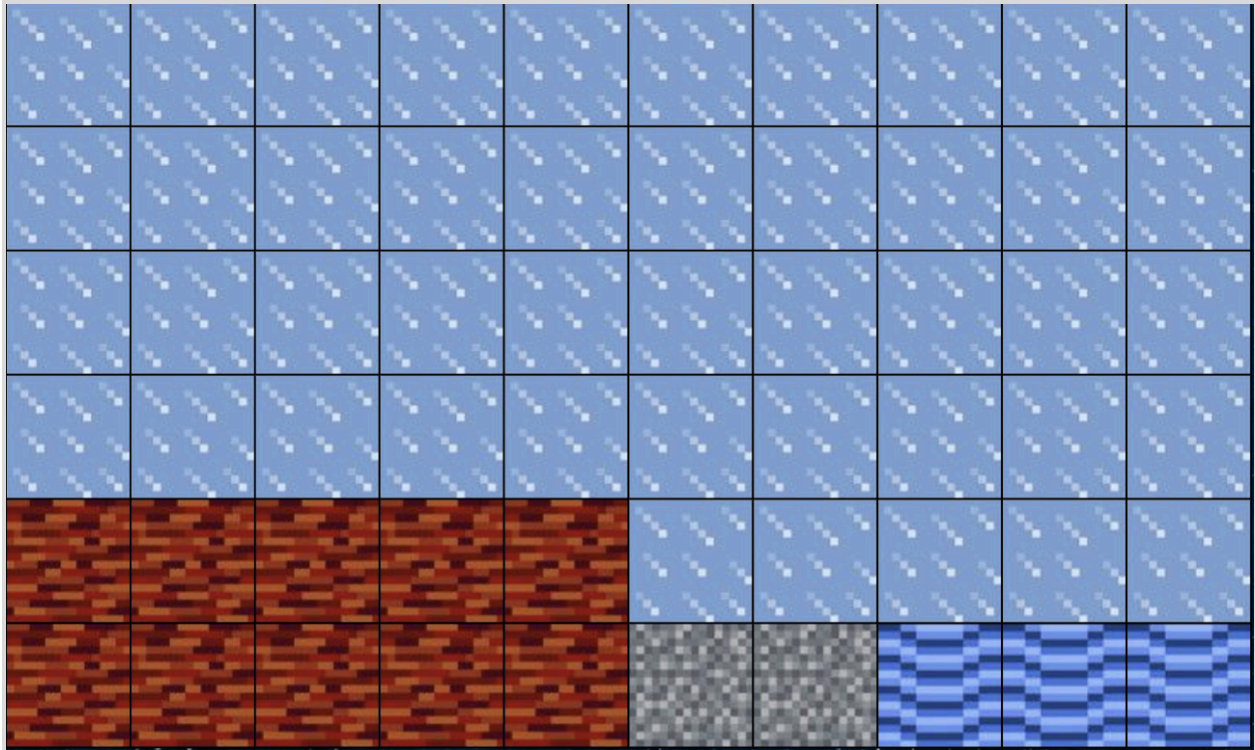
Um mapa é uma grelha ou matriz de peças de terreno. Cada terreno tem diferentes propriedades.

```
type Mapa = [[Terreno]
data Terreno = Ar | Agua | Terra | Pedra
```

Apresenta-se abaixo um exemplo de um mapa simples com 6 linhas e 10 colunas:

```
m = [[Ar,Ar,Ar,Ar,Ar,Ar,Ar,Ar,Ar,Ar]
      , [Ar,Ar,Ar,Ar,Ar,Ar,Ar,Ar,Ar,Ar]
      , [Ar,Ar,Ar,Ar,Ar,Ar,Ar,Ar,Ar,Ar]
      , [Ar,Ar,Ar,Ar,Ar,Ar,Ar,Ar,Ar,Ar]
      , [Terra,Terra,Terra,Terra,Terra,Ar,Ar,Ar,Ar,Ar]
      , [Terra,Terra,Terra,Terra,Terra,Pedra,Pedra,Agua,Agua,Agua]
      ]
```

Visualmente, o mapa `m` corresponde à seguinte figura:



Estado do jogo

Considere a seguinte imagem que representa graficamente um estado do jogo:



Mais concretamente, o estado do jogo consiste num mapa e em informação adicional sobre minhocas e objetos, e é representado pelo seguinte formato:

```
data Estado = Estado
  { mapaEstado :: Mapa
  , objetosEstado :: [Objeto]
  , minhocasEstado :: [Minhoca]  }
```

O estado de exemplo corresponde portanto a acrescentar objetos e minhocas ao mapa de exemplo, e pode ser representado pelo valor:

```
e = Estado { mapaEstado = m, objetosEstado = os, minhocasEstado = ms }
```

Em particular, temos 2 objetos e 2 minhocas (cujas cores das variáveis correspondem às cores que identificam minhocas na figura):

```
o1 = Barril { posicaoBarril = (3,0), explodeBarril = False }
o2 = Disparo { posicaoDisparo = (3,3), direcaoDisparo = Oeste, tipoDisparo = Bazuca, tempoDisparo = Nothing, donoDisparo = 0 }
os = [o1,o2]
m1 = Minhoca { posicaoMinhoca = Just (3,4), vidaMinhoca = Viva 100,
jetpackMinhoca = 1, escavadoraMinhoca = 1, bazucaMinhoca = 1, minaMinhoca = 1, dinamiteMinhoca = 1 }
m2 = Minhoca { posicaoMinhoca = Just (2,0), vidaMinhoca = Viva 100,
jetpackMinhoca = 1, escavadoraMinhoca = 1, bazucaMinhoca = 1, minaMinhoca = 1, dinamiteMinhoca = 1 }
ms = [m1,m2]
```

Pode consultar os tipos de dados completos, incluindo o estado de objetos e minhocas, na [documentação do projeto](https://haslab.github.io/Teaching/L11/2526/viewer.html). Pode visualizar graficamente este e outros estados que defina em <https://haslab.github.io/Teaching/L11/2526/viewer.html>.

Tarefa 1 - Validar estados

O objectivo desta tarefa é implementar uma função de validação de estados do jogo.

Estados válidos

Para todas as tarefas do trabalho prático, só vão ser considerados estados válidos, que estejam bem formados e respeitem as regras do jogo. Um estado é válido quando:

1. O mapa é válido, i.e. satisfaz os seguintes critérios:

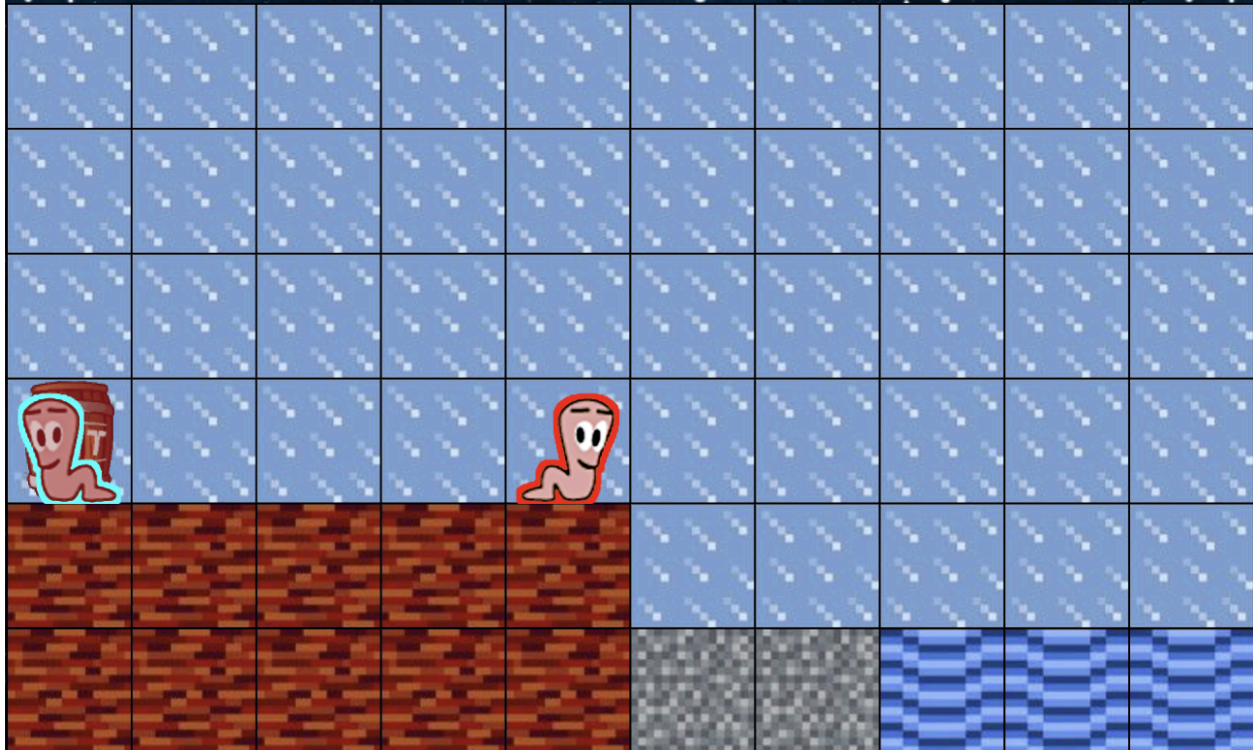
- a. Não vazio;
 - b. Denota corretamente uma grelha, i.e. tem o mesmo número de colunas em todas as linhas.
 - c. Contém terrenos, que podem ser do tipo Ar/Água (não opacos) ou do tipo Terra/Pedra (opacos);
2. Cada objeto é válido, o que se verifica quando:
- a. Tem uma posição válida (dentro dos limites do mapa) e livre no mapa (que não corresponde a nenhum terreno opaco).
 - i. Uma exceção será feita para disparos de bazuca, que podem “temporariamente perfurar” terrenos opacos, mas apenas na superfície, i.e., apenas se a posição anterior dada a direção do projétil não for também um terreno opaco.
 - b. A posição:
 - i. De um barril: não se encontra ocupada por um barril (que não o próprio objeto) ou por uma minhoca.
 - ii. De uma mina ou dinamite: não se encontra ocupada por um barril.
 - iii. De outro objeto: não requer nenhuma validação adicional.
 - c. Se for um disparo:
 - i. Não pode ser um disparo de jetpack ou de escavadora, já que estas armas terão efeito instantâneo no estado do jogo e não criarão disparos.
 - ii. O tempo do disparo tem que ser coerente com o tipo de arma:
 - 1. Bazuca: sem tempo
 - 2. Mina: sem tempo, ou um inteiro entre 0 e 2
 - 3. Dinamite: um inteiro entre 0 e 4
 - iii. O dono do disparo tem que ser um índice válido na lista de minhocas. O mesmo dono não pode ter simultaneamente mais do que um disparo de cada tipo.
3. Cada minhoca é válida, o que se verifica quando:
- a. Tem uma posição válida e livre no mapa (que não contém nenhum terreno opaco), ou opcionalmente nenhuma posição.
 - b. A sua posição, se existir, é também livre no estado (além de ser livre no mapa, não se encontra ocupada por um objeto de barril ou por outra minhoca).
 - c. Quando não tem posição ou se encontra numa posição em que o terreno é água, a minhoca tem que estar obrigatoriamente morta.
 - d. Quando viva, a vida da minhoca é um inteiro entre 0 e 100.
 - e. A quantidade de munições das diversas armas é um número inteiro maior ou igual a 0.

Funções a implementar

O objetivo desta tarefa é definir a função:


```
validaEstado :: Estado -> Bool
```

que decide se um estado é válido ou não, retornando um valor Booleano. Por exemplo, deverá ser verdade que, para o estado de exemplo, `validaEstado e == True`. Quando um estado for inválido, o visualizador de estados irá assinalar a vermelho alguns componentes. Por exemplo, no estado abaixo a minhoca azul e o barril encontram-se sobrepostos na mesma posição:



Poderá consultar a validade de um estado no visualizador no botão com o símbolo . Consulte a [documentação online da Tarefa 1](#) para mais detalhes.

Tarefa 2 - Efetuar jogadas

O objectivo desta tarefa é, dada uma descrição do estado do jogo e uma jogada de uma minhoca, determinar o efeito dessa jogada no estado do jogo.

Definições auxiliares

- Dizemos que uma minhoca/objeto se encontra “no ar” (se o terreno atual for do tipo Ar) ou “na água” (se o terreno atual for do tipo Agua) quando a posição inferior se encontra livre no estado. Caso contrário, dizemos que se encontra “no chão”.

Jogadas

Uma jogada consiste numa movimentação numa dada direção ou no disparo de uma arma numa dada direção, de acordo com o seguinte tipo:

```
data Jogada = Dispara TipoArma Direcao | Move Direcao
```

O processamento de uma jogada deverá ser efetuado de acordo com as seguintes regras:

- **Movimentar:**
 - Para qualquer movimentação inválida (segundo as regras seguintes), a minhoca fica na mesma posição.
 - Uma minhoca só se pode movimentar se estiver viva.
 - A posição do estado para onde se movimenta tem que estar livre.
 - Quando uma minhoca se movimenta para cima (norte/nordeste/noroeste), a minhoca salta (move-se nessa direção) se se encontrar no chão, caso contrário o movimento é inválido.
 - Quando uma minhoca se encontra no ar, não se pode movimentar.
 - Quando uma minhoca se move para uma posição inválida (fora do mapa), esta morre e fica sem posição.
 - Quando uma minhoca se move para uma posição de água, esta morre e fica na nova posição.
- **Disparar:**
 - Quando uma minhoca dispara uma arma tem que estar viva e ter munições suficientes (>0), e gasta 1 unidade de munição com o disparo.
 - Uma minhoca só pode disparar uma arma se não existir um objeto do tipo de disparo no estado atual para a mesma arma pertencente à mesma minhoca.
 - Um disparo de:
 - Jetpack: permite que a minhoca se mova em qualquer direção, desde que a posição de destino esteja livre no estado.
 - Escavadora: destrói a posição de destino se for um terreno de Terra, e movimenta a minhoca para a nova posição livre nessa direção.
 - Bazuca: é colocado na posição de destino, sem tempo e com a direção do disparo.
 - Mina: é colocado na posição de destino se estiver livre, caso contrário na posição atual da minhoca, sem tempo e na direção do disparo.
 - Dinamite: é colocado na posição de destino se estiver livre, caso contrário na posição atual da minhoca, com tempo igual a 4 e na direção do disparo.
 - Objetos colocados fora do mapa são eliminados.

Note que os efeitos de disparar armas só criam novos objetos do tipo disparo ou alteram o estado do jogo de forma imediata. A progressão dos objetos do tipo disparo dependerá da noção de tempo do jogo e será tratada na Tarefa 3.

Funções a implementar

O objectivo desta tarefa é definir a função

```
efetuaJogada :: NumMinhoca -> Jogada -> Estado -> Estado
```

que efetua uma jogada para uma determinada minhoca (dada pelo seu índice na lista de minhocas) sobre um determinado estado, retornando o novo estado após a jogada ser efetuada. Caso a jogada seja inválida, i.e., não cumpre uma ou mais das regras definidas acima ou dá origem a um estado inválido, o estado não deve ser alterado. Consulte a [documentação online da Tarefa 2](#) para mais detalhes.

Por exemplo, dado o estado `e` do exemplo anterior, e a jogada `Movimenta Nordeste` para a minhoca `1`, a função `efetuaJogada 1 (Movimenta Nordeste) e` deveria retornar o estado `e'`

```
e' = Estado { mapaEstado = m, objetosEstado = os, minhocasEstado =  
  [m1,m2']}  
m2' = Minhoca { posicaoMinhoca = Just (1,1), vidaMinhoca = Viva 100,  
  jetpackMinhoca = 1, escavadoraMinhoca = 1, bazucaMinhoca = 1, minaMinhoca =  
  1, dinamiteMinhoca = 1 }
```

Este novo estado pode ser visualizado na seguinte imagem.

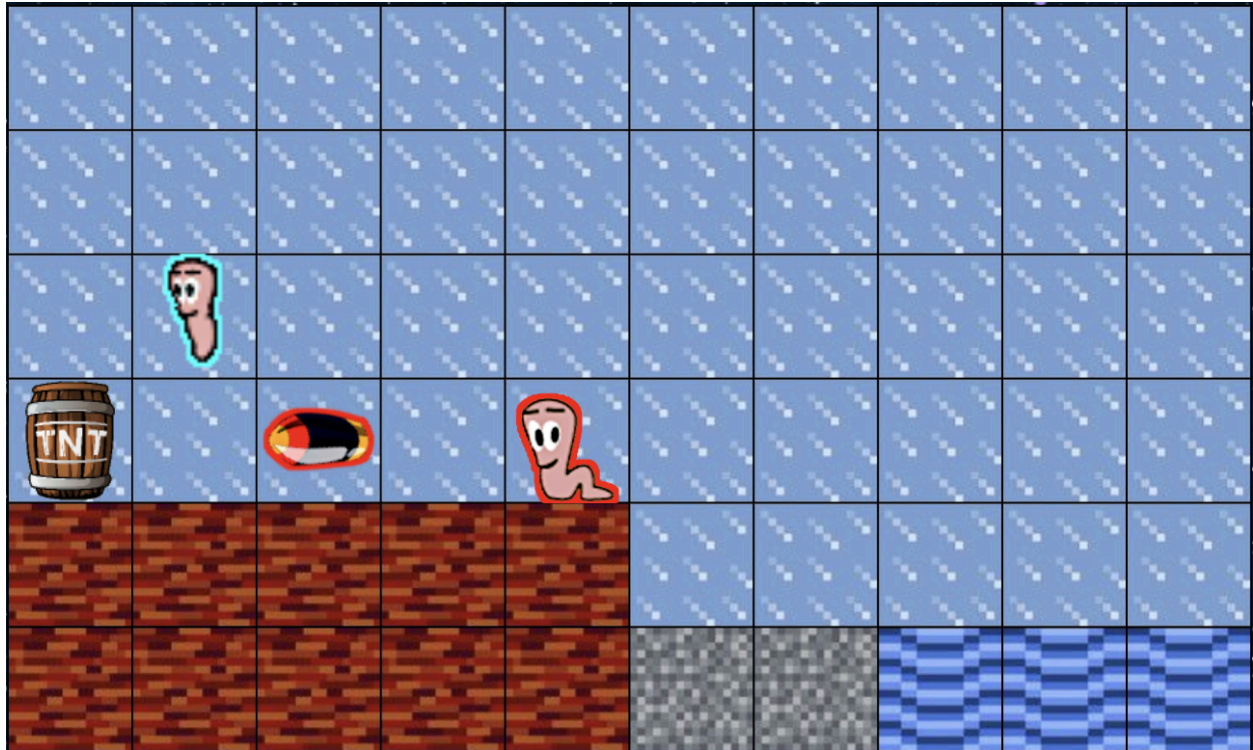


Tarefa 3 - Reagir à passagem do tempo

O objectivo desta tarefa é **calcular o efeito da passagem de um instante de tempo** do jogo sobre um estado do jogo. Por exemplo, dado o estado **e'**, o próximo estado é o seguinte estado **e''**:

```
e'' = Estado { mapaEstado = m, objetosEstado = [o1,o2'], minhocasEstado =
[m1,m2'']}
o2' = Disparo { posicaoDisparo = (3,2), direcaoDisparo = Oeste, tipoDisparo
= Bazuca, tempoDisparo = Nothing, donoDisparo = 0 }
m2'' = Minhoca { posicaoMinhoca = Just (2,1), vidaMinhoca = Viva 100,
jetpackMinhoca = 1, escavadoraMinhoca = 1, bazucaMinhoca = 1, minaMinhoca =
1, dinamiteMinhoca = 1 }
```

Este novo estado pode ser visualizado na seguinte imagem.

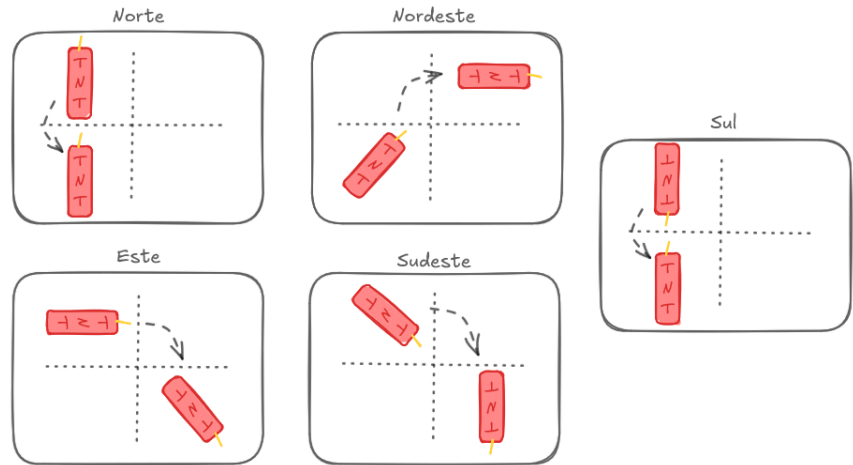


Em particular, a passagem de um instante de tempo deve ter os seguintes efeitos sobre o estado do jogo:

- Minhocas:
 - As minhocas estão sujeitas ao efeito da gravidade. Cada minhoca que esteja no ar ou na água cai para a posição inferior. Se a nova posição for água e a minhoca estiver viva, ela morre. Se a nova posição for inválida, a minhoca fica sem posição e morre.
- Objetos:
 - Um objeto do tipo barril que esteja prestes a explodir, ou um objeto do tipo disparo que tenha tempo igual a 0 ou seja uma bazuca numa posição do estado não livre, é removido do estado e explode, com uma explosão de acordo com as regras definidas abaixo. A explosão de dinamite tem diâmetro 7, ao passo que as explosões de barril/bazuca têm diâmetro 5 e de mina diâmetro 3.
 - Os objetos (que não tenham explodido) estão sujeitos às seguintes leis da física:
 - Um objeto do tipo barril que se encontre no ar ou na água, ao invés de cair para a posição inferior, fica na mesma posição, mas passa para o estado em que está prestes a explodir.
 - Um objeto do tipo bazuca avança para a posição seguinte, de acordo com a sua direção. Se a nova posição for inválida, é removida.
 - Um objeto do tipo dinamite que se encontre no ar executa uma parábola dada por mover-se na sua direção (Este ou Oeste) e muda a sua direção rodando 45° no mesmo sentido. Se tiver direção para Norte/Sul (sem componentes Este ou Oeste), cai na vertical com direção para sul mas

fica a apontar para Norte. Se a nova posição para onde se move não estiver livre no estado, então a dinamite cai na vertical apontando para Norte.

- O seguinte diagrama exemplifica algumas destas situações:



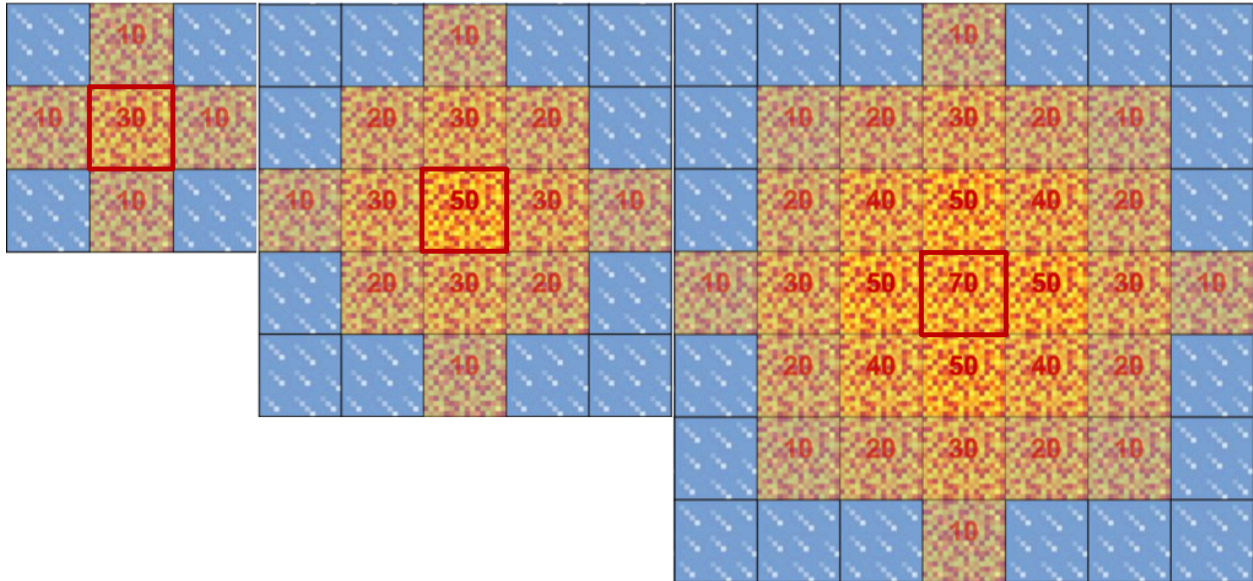
- Um objeto do tipo mina que se encontre na água ou no ar sofre o efeito da gravidade na vertical, isto é, cai para a posição inferior e a sua direção passa a apontar para norte.
- Um objeto do tipo mina/dinamite que se encontre no chão fica imóvel mas a sua direção passa a apontar para norte.
- Os objetos que tenham tempo **n** maior do que 0, seguem a sua contagem decrescente e atualizam o seu tempo para **n-1**.
- Objetos do tipo mina que não tenham tempo são ativados (iniciam contagem decrescente com tempo igual a 2) se uma minhoca que não o dono estiver na sua área de explosão.

Explosões

Uma explosão com diâmetro d afeta um círculo (aproximado em quadrículas do mapa) de $d \times d$ posições à sua volta. O dano da explosão é calculado da seguinte forma:

- A posição da explosão sofre $d \times 10$ de dano;
- As posições a Oeste/Este/Norte/Sul sofrem $(d-2) \times 10$ de dano;
- As posições a Noroeste/Nordeste/Sudeste/Sudoeste sofrem $(d-3) \times 10$ de dano;
- E assim sucessivamente para posições mais distantes, enquanto o dano for >0 .

As imagens em baixo exemplificam explosões de diâmetro 3, 5 e 7, com centro na posição assinalada a vermelho.



Funções a implementar

O objectivo desta tarefa é definir a função

```
avancaEstado :: Estado -> Estado
```

cujo objectivo é avançar um dado estado do mapa exatamente um instante de tempo do jogo. Para organizar melhor esta tarefa, a função **avancaEstado** já vem pré-definida, construída à custa de 3 funções mais simples, que se espera que os alunos definam:

```
type Dano = Int
type Danos = [(Posicao,Dano)]

avancaMinhoca :: Estado -> NumMinhoca -> Minhoca -> Minhoca
avancaObjeto :: Estado -> NumObjeto -> Objeto -> Either Objeto Danos
aplicaDanos :: Danos -> Estado -> Estado
```

Nomeadamente:

1. A função **avancaMinhoca** recebe o estado global e o índice de uma minhoca, e atualiza o estado dessa minhoca;
2. A função **avancaObjeto** recebe o estado global e o índice de um objeto, e atualiza o estado desse objeto ou, caso este seja destruído, retorna uma lista de danos causados por explosões desse objeto;
3. A função **aplicaDanos** atualiza um estado, aplicando uma lista de danos.

Em particular, a função **avancaEstado** trata de invocar estas 3 funções pela ordem definida acima. Consulte a [documentação online da Tarefa 3](#), para mais detalhes.

Sistema de *Feedback*

O projecto inclui também um [Sistema de Feedback](#), que visa incentivar boas práticas no desenvolvimento de software e fornecer informação personalizada a cada grupo. Esta página *web* permite aos alunos pedir feedback relativo ao seu trabalho de grupo e fornece, nomeadamente:

- Resultados de testes unitários, comparando a solução do grupo com um oráculo (solução ideal) desenvolvido pelos docentes;
- Visualizadores gráficos de casos de teste utilizando as soluções do grupo e o oráculo.

Para receber *feedback* sobre as tarefas e qualidade dos testes, devem ser declaradas no ficheiro de testes correspondente a cada tarefa as seguintes listas de testes:

```
testesT1 :: [Estado]
testesT2 :: [(NumMinhoca,Jogada,Estado)]
testesT3 :: [Estado]
```

Como exemplo, podem-se utilizar os exemplos acima definidos como casos de teste:

```
testesT1 = [e]
testesT2 = [(1,Movimenta Nordeste,e)]
testesT3 = [e']
```

Estas definições podem ser então colocadas no ficheiro correspondente a cada tarefa para que sejam consideradas pelo Sistema de *Feedback*. Note que uma maior quantidade e diversidade de testes garante um melhor *feedback* e ajudará a melhorar o código desenvolvido. Relembramos também que, excepto na Tarefa 1, as restantes tarefas só deverão considerar estados válidos: não têm que considerar estados inválidos como testes para as Tarefas 2 e 3.

Para obter o *feedback* de cada tarefa, deve executar localmente no seu projeto o comando (em que **N** é o número da tarefa):

```
cabal run tN-feedback
```

O comando irá compilar o código da tarefa juntamente com os testes, e deverá abrir no seu browser uma página web com uma listagem do feedback por cada teste.

Entrega e Avaliação

A data limite para conclusão de todas as tarefas desta primeira fase é **9 de Novembro de 2025 às 23h59m59s (Portugal Continental)** e a respectiva avaliação terá um peso de 50% na nota final do projeto. A submissão será feita automaticamente através do Git: nesta data será feita uma cópia do repositório de cada grupo, sendo apenas considerados para avaliação os programas e demais artefactos que se encontrem no repositório nesse momento. O conteúdo dos repositórios será processado por ferramentas de detecção de plágio e, na eventualidade de serem detectadas cópias, estas serão consideradas fraude dando-se-lhes tratamento consequente.

Para além dos programas Haskell relativos às 3 tarefas, será considerado parte integrante do projeto todo o material de suporte à sua realização armazenado no repositório Git do respectivo grupo (código, documentação, ficheiros de teste, etc.). A utilização das diferentes ferramentas abordadas na UC (como Haddock, Git, etc.) deve seguir as recomendações enunciadas nas respectivas sessões laboratoriais. A avaliação desta fase do projecto terá em linha de conta todo esse material, atribuindo-lhe os seguintes pesos relativos:

Componente	Peso
Avaliação automática da Tarefa 1	20%
Avaliação automática da Tarefa 2	20%
Avaliação automática da Tarefa 3	20%
Qualidade do código	15%
Qualidade dos testes	10%
Documentação do código usando o Haddock	10%
Utilização do git e estrutura do repositório	5%

A avaliação automática será feita através de um conjunto de testes que não serão revelados aos grupos. A avaliação qualitativa incidirá sobre aspectos de qualidade de código (por exemplo, estrutura do código, elegância da solução implementada, etc.), qualidade dos testes (quantidade, diversidade e cobertura dos mesmos), documentação (estrutura e riqueza dos comentários) e bom uso do Git como sistema de controle de versões.